

Reinforcement Learning

От своего backpropagation к обучению агентов

Отправная точка: что уже умеет nn_lib

- MLP: линейные слои, ReLU, softmax.
- Backpropagation: производная loss проходит через сеть.
- Классификация: метка y задаёт направление $p - y$.

Новый вопрос

В игре никто не сообщает правильное действие. Есть только последствия и награды. Откуда взять loss?

Цель: вывести первый алгоритм, связать его с библиотекой и спроектировать эксперимент на PyTorch с поддержкой GPU. Расширять учебную nn_lib обязательно.

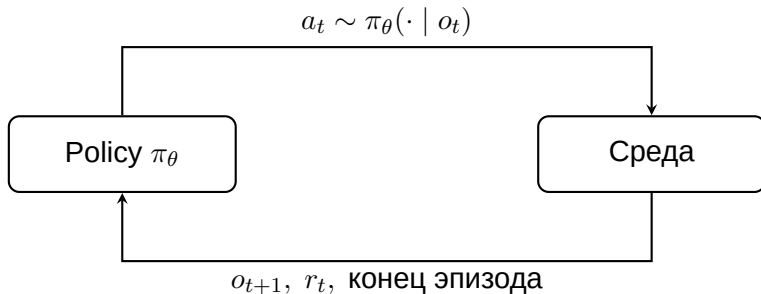
Твоя идея почти совпадает с REINFORCE

- 1 Общая сеть: да, одна policy для многих копий среды.
- 2 Вероятности действий: да, softmax и случайная выборка.
- 3 Градиент действия: точнее, $\nabla_{\theta} \log \pi_{\theta}(a | s)$.
- 4 Обратная связь: весом служит будущий return, позднее — advantage, а не только знак награды.

Что хранить

Проще накапливать наблюдения, действия и награды. Градиенты пересчитать после сбора, сохранив те же параметры policy.

Цикл взаимодействия



Здесь r_t получена **после** действия a_t . Один эпизод: T действий, индексы от 0 до $T - 1$.

State s_t Полное состояние мира.

Observation o_t То, что доступно агенту.

Markov property Текущее состояние и действие определяют распределение следующего состояния и награды.

Позиции недостаточно, если есть инерция: нужна скорость. При ограниченном обзоре могут понадобиться история или recurrent policy.

Упрощение первого проекта

Наблюдение содержит всё необходимое. Ниже пишем s_t . При конечном лимите ходов включаем оставшееся время.

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k, \quad J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right].$$

γ — discount factor, не learning rate.

Отложенная награда

Награды $(0, 0, 1)$, $\gamma = 0.9$: returns $(0.81, 0.9, 1)$. Последнее действие не получает всю заслугу.

В первом конечном эксперименте берём $\gamma = 1$: максимизируем ожидаемую сумму наград эпизода.

Почему обычной классификации недостаточно

Выбранное действие не становится правильным только потому, что агент его выбрал.

$$L_{\text{CE}} = -\log p_a \quad \implies \quad \frac{\partial L_{\text{CE}}}{\partial z_j} = p_j - \mathbf{1}[j = a].$$

Без оценки результата такой update подкрепляет любые собственные действия.

Идея

Увеличивать вероятность действий с хорошими последствиями; уменьшать её для действий с результатом хуже ожидаемого.

Дифференцируем вероятность траектории

Если динамика среды не зависит от параметров θ :

$$p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Используем $\nabla p = p \nabla \log p$:

$$\nabla J = \mathbb{E} [R(\tau) \nabla \log p_{\theta}(\tau)], \quad \nabla \log p_{\theta}(\tau) = \sum_t \nabla \log \pi_{\theta}(a_t \mid s_t).$$

$R(\tau) = \sum_t \gamma^t r_t$. Производные динамики не нужны: игровой движок может быть чёрным ящиком. Дискретную выборку действия также не дифференцируем.

От всей траектории к reward-to-go

Награды до действия не зависят от него причинно; их вклад в ожидаемый градиент равен нулю.

$$\nabla J = \mathbb{E} \left[\sum_t \gamma^t G_t \nabla \log \pi_{\theta}(a_t | s_t) \right].$$

Следим за соглашениями

Для выбранного discounted objective есть внешний γ^t . При $\gamma = 1$ он исчезает. В практических алгоритмах встречаются другие схемы усреднения; не смешиваем их с этим выводом.

Одна траектория даёт шумную оценку. Собираем batch эпизодов.

REINFORCE: loss для знакомого gradient descent

Для B полных эпизодов при $\gamma = 1$:

$$L_{\text{actor}} = -\frac{1}{B} \sum_{i=1}^B \sum_t \text{stopgrad}(G_{i,t}) \log \pi_{\theta}(a_{i,t} \mid s_{i,t}).$$

- Награды и returns — фиксированные численные веса.
- Минимизируем loss, значит максимизируем ожидаемый reward.
- Делим на число эпизодов: сохраняем сумму вкладов внутри эпизода.
- Это surrogate loss; качество измеряем в новых эпизодах.

Мост к backpropagation: всего один множитель

Для одного действия с весом w :

$$L = -w \log p_a, \quad \frac{\partial L}{\partial z_j} = w(p_j - \mathbf{1}[j = a]).$$

Остальная цепочка backpropagation не меняется.

Проверка знака, без batch-нормировки

$p = (0.2, 0.5, 0.3)$, выбрано второе действие, $w = 2$. Производная $(0.4, -1, 0.6)$. Gradient descent увеличивает выбранный logit. При $w = -2$ направление противоположно.

Необязательно: как выглядел бы update в nn_lib

- `back_propagation`: вместо $p - y$ нужно $w(p - y)$.
- Не передавать wy как метку: $p - wy \neq w(p - y)$.
- Накопить batch и вызвать `apply_grads` один раз. Текущий `train` обновляет веса при каждом вызове.
- Отключить dropout: `predict` и обучение должны пересчитывать одну и ту же policy. Для отладки также отключить L2.
- Считать log probability через стабильный log-softmax.
- Для critic понадобится линейный выход: softmax одного числа всегда равен 1.

Это мост к знакомому коду. Основной проект использует PyTorch autograd.

PyTorch: backprop уже реализован

```
# policy outputs logits; tensors are on the same device
# batch contains B complete episodes, gamma = 1
probs = torch.distributions.Categorical(logits=policy(obs))
loss = -(probs.log_prob(actions) * returns.detach()).sum() / B
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

При сборе: `no_grad()` и `Categorical.sample()`. До обработки batch параметры заморожены.

CPU / GPU

Сеть и batch размещаем на одном device. Конкретный пример и настройка устройства есть в конспекте.

Полный первый training loop

```
repeat:
    freeze policy parameters
    episodes = collect_complete_episodes(B) # sample actions
    zero_grad()
    for episode in episodes:
        G = 0
        for (obs, action, reward) in reverse(episode):
            G = reward + G # gamma = 1
            p = forward(obs) # same policy
            backward_logits(G * (p - one_hot(action)) / B)
    apply_gradients_once()
    discard episodes
```

У среды настоящий конечный горизонт. Returns не пересекают reset. Равномерно выбираем reset seeds, не отбрасываем долгие эпизоды.

Почему сначала храним опыт, а не градиенты

- У каждого действия свой G_t .
- После суммирования градиентов разные веса уже не восстановить.
- Хранить градиент по всем параметрам на каждом шаге дорого.
- Наблюдение, действие и награда позволяют повторить forward/backward.

Условие корректности простого алгоритма

До конца обработки batch веса policy не меняются. После update собираем новые данные.

Baseline: хорошо относительно чего?

$$V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s], \quad \hat{A}_t = G_t - V_{\phi}(s_t).$$

Ожидали 5, получили 2: advantage = -3 , даже при положительном return.

$$\mathbb{E}_{a \sim \pi}[b(s) \nabla \log \pi(a \mid s)] = b(s) \nabla \sum_a \pi(a \mid s) = 0.$$

Фиксированный baseline, не зависящий от выбранного действия, не меняет ожидаемый градиент и может уменьшить дисперсию. Начни со среднего return предыдущих batches.

Actor и critic: две задачи обучения

Actor π_θ Выбирает действие.

Critic V_ϕ Предсказывает будущий return текущей policy.

$$L_{\text{actor}} = -\frac{1}{B} \sum_{i,t} \text{stopgrad}(\hat{A}_{i,t}) \log \pi_\theta(a_{i,t} \mid s_{i,t}),$$

$$L_{\text{value}} = \frac{1}{2N} \sum_{i,t} (V_\phi(s_{i,t}) - \text{stopgrad}(G_{i,t}))^2.$$

N — число переходов. Сначала две отдельные сети. Веса actor вычислить старым critic до обучения на текущем batch.

TD: не обязательно ждать конца эпизода

$$y_t = r_t + \gamma(1 - \text{terminated}_t)V_{\text{old}}(s_{t+1}), \quad \delta_t = y_t - V_{\text{old}}(s_t).$$

- Monte Carlo: фактический полный return; высокая дисперсия.
- Temporal difference: reward плюс оценка продолжения.
- Bootstrapping снижает зависимость от полного исхода, но неточный critic вносит смещение.
- Targets фиксированы при обновлении; δ_t можно использовать как оценку advantage.

Окончание задачи и остановка сбора — разные события

Terminated Задача закончилась: bootstrap равен нулю.

Truncated Внешний обрыв: используем value последнего наблюдения до reset.

Лимит времени

Если 40 ходов — правило игры, это termination; оставшееся время входит в observation. Если остановился только сбор данных, это truncation.

При обоих флагах termination имеет приоритет. REINFORCE без critic сначала использует только полные эпизоды.

$$\hat{A}_t = \delta_t + \gamma \lambda c_t \hat{A}_{t+1}.$$

- λ регулирует компромисс смещения и дисперсии.
- $c_t = 0$ на границе эпизода или rollout batch.
- На внешнем обрыве TD сохраняет bootstrap, но рекурсия advantage не пересекает reset.
- При $\lambda = 0$ остаётся одношаговый TD residual.
- При $\lambda = 1$ и полном эпизоде сумма даёт $G_t - V(s_t)$.

Реализовать после рабочей Monte Carlo версии с baseline.

Почему нельзя бесконечно учиться на старом batch

On-policy: данные собраны policy, градиент которой оцениваем. После update вероятности действий и распределение посещаемых состояний меняются.

- REINFORCE: один batch, один update, новый опыт.
- PPO: несколько epochs на свежем batch с поправкой objective.
- Off-policy методы специально устроены для использования опыта другой policy; пример — DQN с replay buffer.

Обычный многократный train на прежних эпизодах не превращает REINFORCE в PPO.

PPO: следующий этап, а не первый

Сохраняем старые log probabilities и advantages:

$$\rho_t(\theta) = \exp(\log \pi_\theta(a_t | s_t) - \log \pi_{\text{old}}(a_t | s_t)),$$

$$L_{\text{clip}} = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Ограничиваем стимул слишком сильно менять вероятности на старом batch. Clipping не гарантирует жёсткую границу изменения policy. Контролируем KL divergence, entropy и clip fraction.

Практический порядок

REINFORCE → baseline → actor–critic → GAE → PPO.

Где другие направления RL

Policy gradient Непосредственно учим вероятности действий.

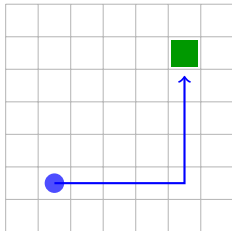
Q-learning / DQN Учим $Q(s, a)$ — ожидаемый return после действия; нужны exploration и Bellman targets.

Actor–critic Совмещаем policy и оценку ценности. PPO и SAC относятся к этому широкому направлению.

Model-based Учим или используем модель динамики и планируем.

Для непрерывного управления возможна Gaussian policy вместо softmax. Это потребует другого log probability и обработки границ действий. Первый проект оставляем дискретным.

Первый проект: сетка 7×7



Разные старт и цель, без препятствий и инерции.

- 4 действия; выход за стену тратит ход.
- Не больше 40 ходов — правило задачи.
- +1 за цель, -0.01 за любой другой ход.
- Observation: координаты агента, цели и время.
- MLP $5 \rightarrow 32 \rightarrow 32 \rightarrow 4$, $\gamma = 1$.

Ручная policy достигает цели максимум за 12 ходов: используем её для проверки среды.


```
reset(seed) -> observation, info  
step(action) -> observation, reward,  
                terminated, truncated, info
```

- Среда владеет физикой, состоянием, reward и reset.
- Тренер владеет сетью, optimizer и batch.
- Action соответствует фиксированному числу physics ticks.
- Рендеринг не влияет на simulation timestep.
- В headless режиме те же правила, больше шагов в секунду.

Сначала один процесс; IPC и batch-команды добавлять по необходимости.

Общая сеть: две разные постановки

Независимые среды

Много копий одной задачи собирают траектории общей policy. Это первый вариант: свои reset, rewards и returns у каждой копии.

Общая арена

Агенты влияют друг на друга. Нужны правила общей/личной награды, наблюдений и распределения заслуги. Возникает multi-agent RL.

В NN есть изменяемые forward-буферы: один объект нельзя одновременно вызывать из разных потоков без изменения архитектуры.

Маршрут реализации с проверками

- 1 Bandit: один шаг, три действия, проверить знак и gradient check.
- 2 Сетка: REINFORCE на полных эпизодах, random baseline.
- 3 Baseline и critic: сравнить при одинаковом бюджете шагов.
- 4 TD и GAE: вручную проверить targets и границы эпизодов.
- 5 Движок: сначала ручной агент, затем обучение без рендера.
- 6 Готовый PPO из Stable-Baselines3; при разборе проверить $\rho = 1$ до update и оба знака advantage.
- 7 Независимые арены; затем взаимодействующие агенты.

Каждый этап должен работать отдельно до добавления следующего.

Готовый стек для проекта

- 1 PyTorch: короткий учебный REINFORCE, готовые autograd и Adam.
- 2 Gymnasium: первая среда и стандартный контракт.
- 3 Stable-Baselines3: готовый PPO для более сложной задачи.
- 4 Godot RL Agents: готовый мост между движком и Python.

TorchRL — альтернатива для более детального управления компонентами. Свой PPO, IPC и GPU kernels не обязательны для первого проекта.

Разделение ответственности

Python обучает сеть; движок симулирует мир и показывает поведение.

GPU ускоряет сеть, но не весь эксперимент

- На GPU: батчевый forward, backward, optimizer.
- Физика обычной Godot-среды автоматически туда не переносится.
- Маленькая MLP может работать быстрее на CPU из-за накладных затрат.
- Больше независимых сред помогает собирать batch; скорость ограничивается также физикой и обменом данными.

Измеряем steps/s, время сбора и время update на CPU и GPU. Для PPO с MLP документация SB3 рекомендует рассмотреть CPU.

Выбор оборудования следует из измерений

Поддержка GPU в стеке есть; обещать ускорение для любой среды нельзя.

Что считать успехом эксперимента

Предлагаемый критерий для сетки:

- 90% успехов на 500 evaluation seeds.
- Критерий выполнен у 4 из 5 независимо обученных моделей.
- Первый бюджет — миллион взаимодействий, затем анализ.
- Отдельно измерять stochastic и greedy policy.

Логировать success rate, return, длину эпизода, entropy, gradient norm и число шагов среды. Сохранять seed, конфигурацию, checkpoint и replay.

Не путать метрики

Падение actor loss не доказывает улучшение поведения.

Типичные поломки и что проверять

Не учится даже bandit Знак update, вес всего $p - y$, sampling.

Учится только одному действию Entropy, размер шага, распределение наград, exploration.

Награда растёт, цель не достигается Агент оптимизирует reward, а не наше намерение: проверить обходные стратегии.

Ломается после reset Returns/GAE пересекли эпизоды или bootstrap взят от нового стартового состояния.

Работает только один seed Оценить разброс, бюджет и настройки.

Не сходится в движении Сначала проверить observation, физические шаги и ручную policy.

Вопросы для самопроверки

- 1 Почему нужен $\log \pi(a | s)$, а не градиент действия?
- 2 Почему награда +2 может уменьшить вероятность действия?
- 3 Почему нельзя передать $w \cdot \text{one_hot}(a)$ в старый train?
- 4 Почему softmax с одним выходом не годится для critic?
- 5 Когда timeout требует bootstrap?
- 6 Почему общий replay buffer не делает REINFORCE off-policy?

Выводы, ответы и подробная спецификация проекта:
`presentations/rl/README.md`.

- 1 Sutton & Barto, Reinforcement Learning: An Introduction. Главы 2–3, 6, 13.
- 2 Spinning Up: Intro to Policy Optimization. Вывод policy gradient и reward-to-go.
- 3 Gymnasium: Handling Time Limits. Termination, truncation и bootstrap.
- 4 Schulman et al., Generalized Advantage Estimation.
- 5 Schulman et al., Proximal Policy Optimization Algorithms.

Сначала вывести и реализовать REINFORCE; статьи про GAE и PPO читать после первого работающего агента.

- PyTorch: Categorical — sampling и log probability.
- Stable-Baselines3: PPO — API, параметры, CPU/GPU.
- Godot RL Agents — интеграция среды с Python.
- TorchRL: PPO tutorial — альтернативный путь к компонентам алгоритма.

Версии зависимостей фиксировать при создании проекта. Конспект содержит примеры PyTorch update и подключения готового PPO.